Okay plain English.

Today a home health agency gets a call from a hospital at 7 PM. "We're discharging a 72-year-old woman tomorrow morning, she just had a hip replacement, she needs a nurse to come to her home 3 times a week. She has Medicare. Can you take her?"

Nobody's in the office. The call goes to voicemail. The discharge planner doesn't wait — she calls the next agency on her list. That agency picks up, says yes. Your agency lost a patient. That's lost revenue. And this happens over and over — agencies lose more than a third of all referrals that come to them.

Even when someone IS in the office and picks up the phone, they can't give an answer on the spot. They have to manually check — do we serve her zip code? Do we accept her insurance? Do we have a nurse with orthopedic experience available this week? Is her insurance going to need any special paperwork? They put the caller on hold, dig through spreadsheets and the EMR, and call back 2 hours later. By then the planner already placed the patient with another agency.

Same thing with faxes. A 50-page referral packet comes in. Someone has to sit there for an hour reading through it, pulling out the patient's name, diagnosis, insurance info, medications, doctor's orders — typing it all into the system manually. Then figuring out what's missing. Then calling the hospital back to get the missing documents. Then calling the patient's family to confirm. Days pass.

What we're building solves this:

The phone rings at 7 PM — our AI agent picks up. It has a real conversation with the discharge planner, collects all the patient details, instantly checks the database to see if we serve that area, accept that insurance, and have the right caregiver available — and gives an answer right there on the call. "Yes, we can take her. We have a nurse available. Send us the face-to-face documentation and we'll have someone there by tomorrow afternoon." Three minutes. Done.

A fax comes in — our system reads it automatically, pulls out all the important information, catches errors, flags what's missing, checks if we can take this patient, and if we can, immediately calls the hospital to collect the missing paperwork and calls the family to schedule the first visit. What used to take days now takes hours.

A worried daughter calls at midnight — our agent picks up, talks to her compassionately, collects what she knows about her mom's situation, tells her whether we can likely help, and assures her that a coordinator will follow up first thing in the morning with next steps. That daughter doesn't have to call 5 agencies hoping someone answers.

In one sentence: We make sure the agency never misses a patient because nobody picked up the phone, and never loses a patient because they were too slow to respond.

That's it. Speed wins patients. This is speed.

Data

Patient Data

- Name
- Date of birth
- Gender
- Address (street, city, state, zip code)
- Phone number
- Emergency contact name and phone
- Primary language spoken

Clinical Data

- Primary diagnosis (and ICD-10 code)
- Secondary diagnoses
- Surgical procedures performed
- Medications (name, dosage, frequency)
- Allergies
- Functional limitations (mobility, self-care ability)
- Homebound status (yes/no, reason)
- Admission date to hospital
- Discharge date from hospital

Physician Data

- Ordering physician name
- Physician NPI number
- Physician phone and fax
- Physician specialty
- Face-to-face encounter date
- Signed physician orders (what care is ordered, visit frequency, duration)

Insurance Data

- Payer name (Medicare, Medicaid, Humana, Aetna, etc.)
- Plan type (Part A, Part B, Medicare Advantage, PPO, HMO)
- Member ID
- Group number
- Policy effective date
- Prior authorization required (yes/no)
- Authorization number (if already obtained)

Referral Source Data

- Referring facility name (hospital, SNF, physician office)
- Referring facility type

- Discharge planner or social worker name
- Their phone, fax, email
- EHR system they use
- Referral date and time

Care Request Data

- Service types needed (skilled nursing, physical therapy, occupational therapy, speech therapy, home health aide)
- Visit frequency requested (3x/week, daily, etc.)
- Duration of care (30 days, 60 days, ongoing)
- Urgency level (routine, urgent, start of care within 24 hours)
- Special instructions from physician

Agency Configuration Data (our own data)

- Service areas (list of zip codes served)
- Insurance contracts (which payers and plans accepted)
- Service types offered
- Operating hours
- Fax number, phone number

Caregiver Roster Data

- Caregiver name
- Caregiver type (RN, LPN, CNA, PT, OT, ST, HHA)
- Certifications held (wound care, IV therapy, pediatric, cardiac, orthopedic, diabetes education, etc.)
- Certification expiry dates
- Service zip codes (where they're willing to work)
- Availability schedule (days, hours)
- Languages spoken
- Current patient load
- Max patient capacity
- Status (active, on leave, suspended)

Knowledge/Reference Data

- ICD-10 code table (code, description, hierarchy)
- Diagnosis to service type mapping (hip replacement → skilled nursing + PT)
- Service type to certification mapping (skilled nursing → RN or LPN)
- Payer coverage rules (Medicare Part A → covers skilled nursing, requires homebound status, requires F2F encounter, no prior auth)
- Medication reference (drug names, standard dosage ranges, interactions)
- NPI registry (for physician validation)

Pipeline/Operational Data

- Intake record status (new, in progress, pending documents, eligible, accepted, declined)
- Extracted fields with confidence scores
- Gap list (what's missing)
- Call transcripts
- Follow-up actions (SMS sent, call attempted, document received)
- Timestamps for everything (referral received, eligibility checked, first call made, patient admitted)

Projetc .md

IntakeAI — Intelligent Patient Intake Agent for Home Health Agencies

Hackathon

Event: AI Healthcare Hack NYC **Host:** localhost:nyc × Arya Health (Series A, reimagining the economics of healthcare services) **Sponsor:** Twilio AI Startup Searchlight **Format:** One-day, in-person sprint — 5 hours build, live demo + judging **Requirement:** Must use Twilio for telephony to qualify for sponsor prizes **Prize:** $500/$300/$200 Twilio credits + interview with Arya Health engineering team for top 3 **Team Size:** 4 engineers using Cursor

---

The Problem

Who is the customer?

Home health agencies — companies that employ caregivers (nurses, physical therapists, home health aides) and send them to patients' homes to deliver care after hospital discharge, surgery recovery, or for chronic condition management. Examples: TheKey, Team Select, Thrive Skilled Pediatric Care. These agencies are Arya Health's customers.

What is Arya Health?

Arya Health is a software company (not a care provider). They sell AI-powered digital agents to home health agencies that automate administrative work. Their current product lineup has 5 agents — all caregiver-facing:

1. **Staffing Agent** — scheduling, shift coverage, callout management
2. **Compliance Agent** — license/certification tracking and follow-up
3. **Onboarding Agent** — recruiting and applicant engagement
4. **Payroll Agent** — rate calculations, overtime, retroactive pay
5. **Engagement Agent** — performance management and retention

All 5 agents manage the supply side (caregivers). None of them handle the demand side (patients coming in). The Intake Agent — handling new patient referrals — is Arya's publicly stated next product, announced during their Series A in late 2025, but has not shipped publicly as of July 2026.

What is the intake problem?

When a patient needs home health care, a referral enters the agency through one of these channels:

1. **Hospital discharge planner sends a fax** — dominant channel. A 35-100+ page PDF referral packet containing patient demographics, diagnosis, physician orders, medications, insurance info, discharge notes. Arrives at any hour. Currently sits in a queue until a human reviews it manually.

2. **Hospital discharge planner calls the agency** — they want an immediate yes/no. "Can you take this patient?" If the agency doesn't pick up or takes too long, they call the next agency.

3. **Family member calls** — a worried daughter or spouse. "My mom just had a stroke and she's coming home, I don't know what to do." They don't have clinical details. They need guidance and reassurance.

4. **Physician's office sends a referral** — via fax or phone call with physician orders.

5. **Skilled nursing facility (SNF) refers a patient** — patient stepping down from rehab to home care.

6. **Patient self-refers** — rare for skilled care (needs physician order), more common for personal care/companion care.

Why is this a problem? (Evidence-based)

- Referral conversion rates have declined 13% since 2018, dropping from 77% to 64% by Q2 2025. Agencies lose more than a third of the patients referred to them.
- Home health agencies collectively lose an estimated $200-500 million annually to referral leakage.
- Most loss happens in the first 70 minutes after a referral arrives while the coordinator manually reviews documents and checks systems.
- It takes 70 minutes for intake coordinators to thoroughly review an average referral packet.
- Median time from referral entry to start of care exceeds 69 hours, with 13+ hours spent inside intake processes alone.
- Hospital discharge planners work with 3-5 agencies simultaneously and go with whoever responds first.
- Agencies miss roughly 20% of referrals arriving outside business hours.

- 30% of referrals are rejected due to service offering mismatches that could have been caught instantly.
- One-third of home health episodes have delayed start-of-care visits.
- Incomplete documentation accounts for over 51% of improper payments in home health.

Why speed matters

Medicare's quality metric requires timely initiation of care — best practice is first visit within 48 hours of referral. If the agency is slow, the patient either goes to a competitor or goes without care. Every hour of delay during intake is a patient the agency might lose. The discharge planner is making 3-5 calls simultaneously — whoever responds first wins the patient.

The Solution

What we're building

IntakeAI is an intelligent patient intake agent that ensures a home health agency never misses a patient because nobody picked up the phone, and never loses a patient because they were too slow to respond.

It handles the entire intake workflow from referral received to patient admitted — across voice calls, fax processing, SMS, and email — with real-time eligibility checking, domain-grounded knowledge, and caller personalization.

In plain English

The phone rings at 7 PM — our AI agent picks up. It has a real conversation with the discharge planner, collects all the patient details, instantly checks the database to see if we serve that area, accept that insurance, and have the right caregiver available — and gives an answer right there on the call. "Yes, we can take her. We have a nurse available. Send us the face-to-face documentation and we'll have someone there by tomorrow afternoon." Three minutes. Done.

A fax comes in — our system reads it automatically, pulls out all the important information, catches errors, flags what's missing, checks if we can take this patient, and if we can, immediately calls the hospital to collect the missing paperwork and calls the family to schedule the first visit. What used to take days now takes hours.

A worried daughter calls at midnight — our agent picks up, talks to her compassionately, collects what she knows about her mom's situation, tells her whether we can likely help, and assures her a coordinator will follow up first thing in the morning.

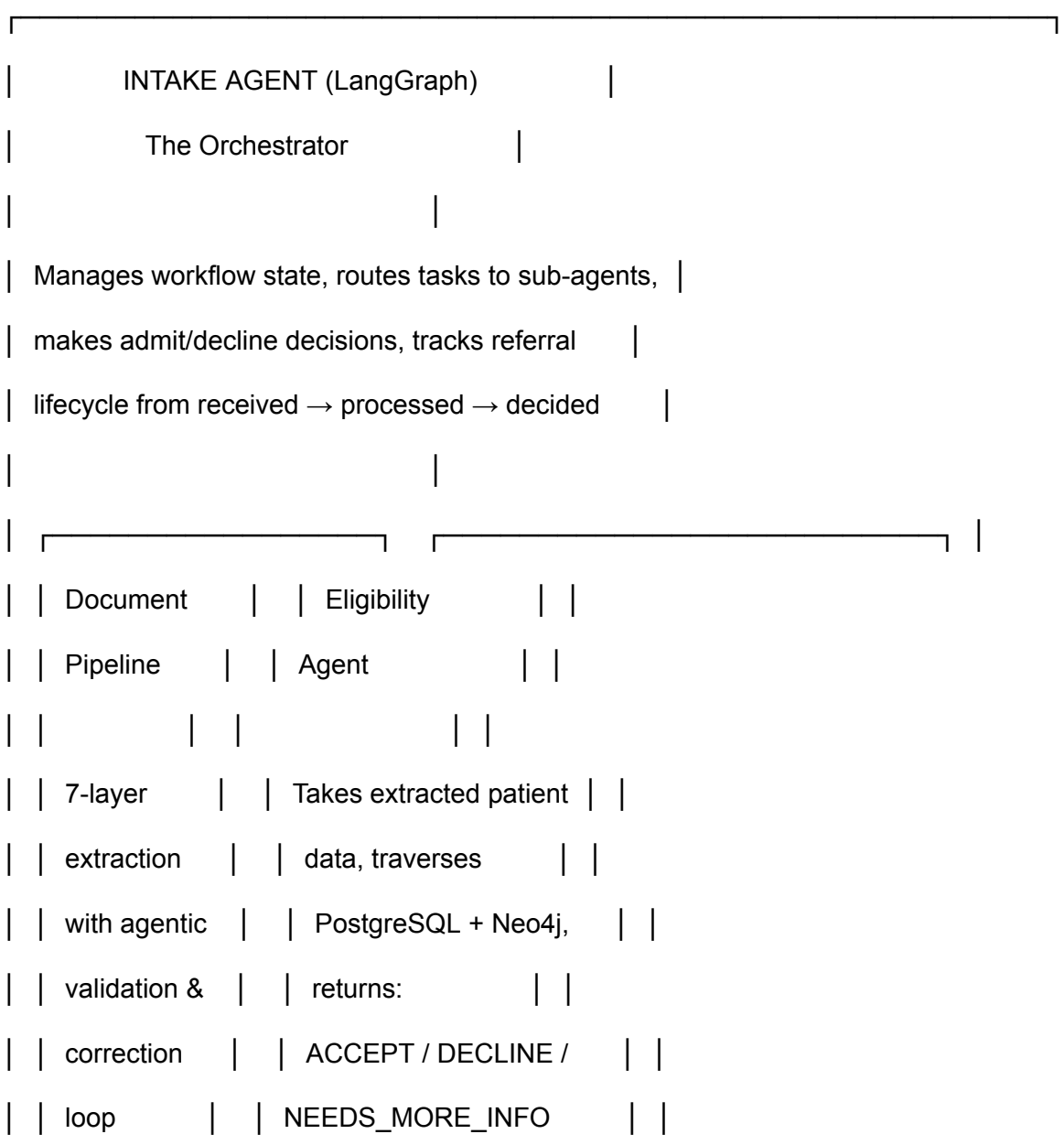
System Architecture

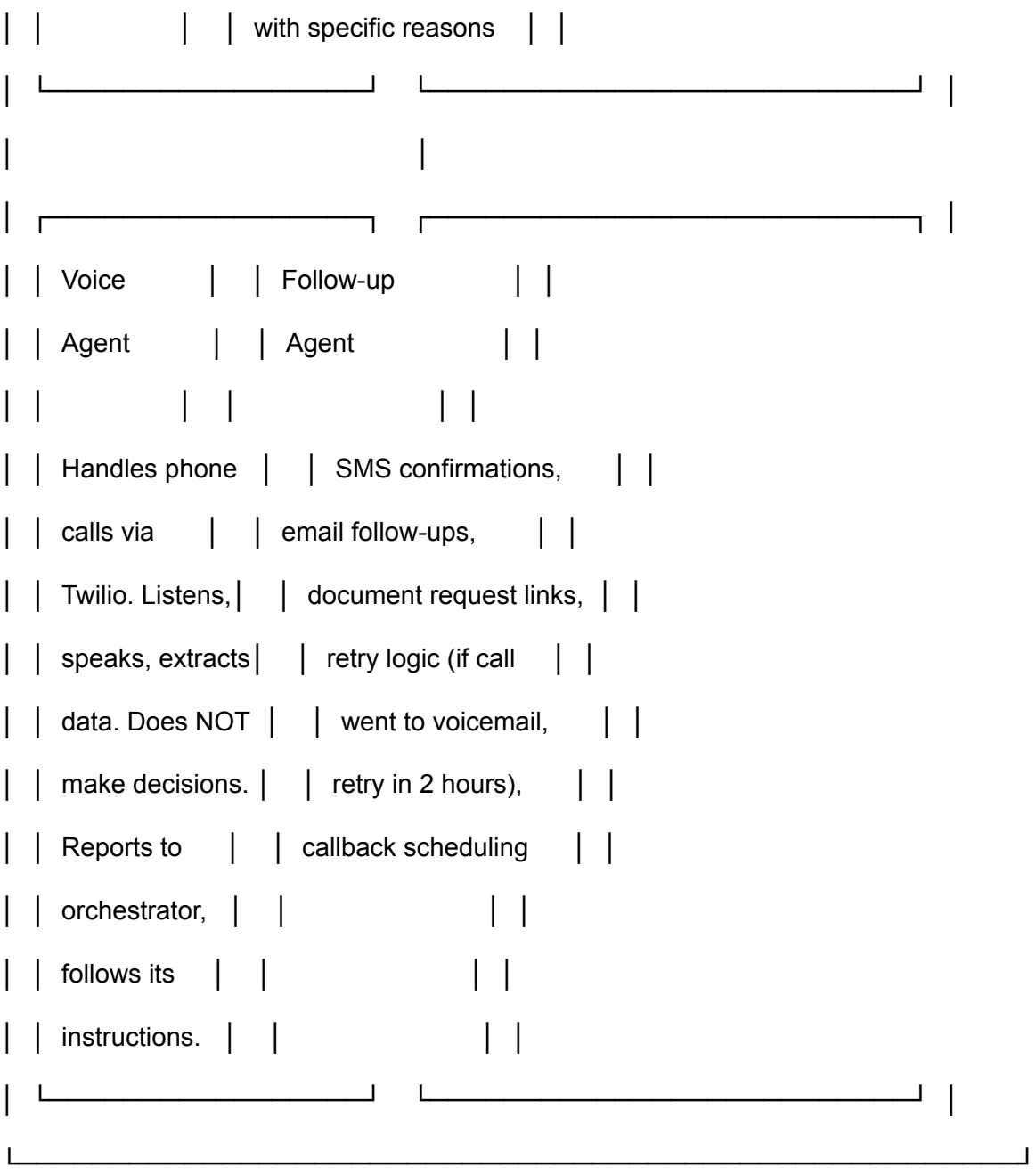
Core Principle

The Intake Agent is the orchestrator — the brain. It doesn't make calls, it doesn't read documents, it doesn't query databases directly. It coordinates specialized sub-agents that each do one thing well.

The Voice Agent is one component inside the Intake Agent. It's the mouth and ears — how the Intake Agent talks to the outside world via phone. The Voice Agent doesn't think for itself. The Intake Agent feeds it everything: what to ask, what to say, when to pause and check eligibility.

Agent Architecture





Voice Agent — Two Layers of Control

The Voice Agent receives instructions from the Intake Agent at two levels:

Layer 1: Static — Conversation Flow (system prompt)

Before any call starts, the Intake Agent assigns a system prompt based on the scenario:

- **Provider mode** — discharge planner or physician calling. Clinical language, efficient, structured. Collects: patient name, DOB, diagnosis, insurance, physician, care type, discharge date, urgency. Doesn't give medical advice. Doesn't confirm admission

without checking with the orchestrator first.

- **Family mode** — family member calling. Compassionate, simple language, no medical jargon. Collects what they know. Sets expectations that a coordinator will follow up. Doesn't promise coverage or availability.

- **Outbound mode** — agent making a call to follow up on a fax referral. Has a specific mission: collect a missing document, verify information, or confirm with patient/family. Knows exactly what gaps to fill.

Layer 2: Dynamic — Real-time Decisions Mid-Call

During a call, the Voice Agent extracts data from what the caller says and passes it to the Intake Agent orchestrator. The orchestrator sends it to the Eligibility Agent for real-time database/graph checks. The result comes back with specific instructions for what the Voice Agent should say next.

Example flow during a live call:

1. Discharge planner says: "Medicare Part A, skilled nursing, post-hip replacement, zip 11201"
2. Voice Agent extracts: zip=11201, insurance=Medicare Part A, service=skilled nursing, diagnosis=hip replacement
3. Voice Agent passes to Intake Agent orchestrator
4. Orchestrator sends to Eligibility Agent
5. Eligibility Agent queries PostgreSQL (do we serve 11201? → yes) → queries Neo4j (hip replacement → ICD M17.11 → requires RN with orthopedic cert) → queries caregiver roster (3 available RNs with that cert in that zip) → checks Medicare coverage (covered, no prior auth, but needs F2F documentation and homebound certification)
6. Eligibility Agent returns: ACCEPT — missing face-to-face encounter note
7. Orchestrator tells Voice Agent: "Tell them we can accept, we have availability within 48 hours, ask for the face-to-face documentation"
8. Voice Agent speaks: "We can accept this referral. We have a nurse available in that area. We'll need the face-to-face encounter documentation — can you send that to our fax?"

This entire loop must complete in 2-3 seconds. If longer, the Voice Agent uses natural filler: "Let me check our availability for that area... one moment."

Document Pipeline (7 Layers + Agentic Review Loop)

Medical referral packets are messy — handwritten physician notes, inconsistent formats across hospitals, mixed document types in one PDF, critical data where a single wrong digit

means a denied claim. The pipeline uses a dual-path extraction approach with an agentic validation-correction loop on every document.

Layer 1: Ingestion & Preprocessing

Fax arrives as PDF/TIFF → convert to standardized format → page-level image cleanup (deskew, denoise, contrast enhancement) → detect if pages are scanned images vs digital text. Medical faxes are notoriously bad quality — smudged, rotated, half-cut pages.

Layer 2: Page Classification

Classify each page in the packet — is this a physician order? Face-to-face encounter note? Discharge summary? Medication list? Insurance card? Lab results? Consent form? Fax cover sheet (junk)? This matters because downstream extractors need to know which document type they're processing. A medication list is extracted differently from an insurance card.

Layer 3: OCR Strategy Router (Dual-Path Extraction)

Path B: Rule-based extraction for clean digital PDFs. Hospital referrals from larger health systems sometimes come as structured digital PDFs. Use Docling to parse text layers, then regex and keyword matching to pull fields. Patient name follows "Patient:" or "Name:". ICD codes follow a known pattern (letter + digits). Insurance member IDs follow payer-specific formats. NPI numbers are always 10 digits. Deterministic, fast, reliable.

Path C: LLM vision extraction for messy image PDFs. Handwritten physician notes, scanned insurance cards at an angle, smudged medication lists. Gemini Flash vision extracts fields directly from the image. The LLM is the primary extractor here because rules can't handle the variation.

Layer 4: Entity Extraction → Raw JSON

Both paths produce a standardized raw JSON output per page with all extracted fields.

Layer 5: Agentic Review Loop

This runs on EVERY document regardless of extraction path. Three agents operate in sequence:

Validation Agent — checks every extracted field:

- Is this a valid ICD-10 code? (lookup against ICD-10 code table)
- Is this NPI number real? (Luhn algorithm check digit validation)
- Does this insurance member ID match the payer's known format?
- Is the date of birth reasonable? (not 150 years old, not future)
- Does discharge date come after admission date?
- Is the zip code a real US zip code?

- Does the medication dosage fall within known clinical ranges? (Metformin 500-2000mg normal, 50000mg is OCR error)

Correction Agent — takes validation failures and reasons about them:

- ICD code "M17.1I" invalid → looks at original image → "I" was actually "1" → corrects to "M17.11" → confidence: high, auto-correct
- Insurance ID "H12O456789" → "O" probably a "0" based on surrounding digits → corrects to "H120456789" → confidence: medium, flag for review
- "Lisinopril 200mg" → standard dosing is 2.5-40mg → could be 20mg or 2.0mg → can't confidently correct → confidence: low, flag for human review, add to gap list for Voice Agent to verify on call

Cross-Reference Agent — checks consistency across pages in the same packet:

- Patient name on discharge summary says "Johnson, Maria" but insurance card says "Maria L. Johnson-Smith" → checks DOB across both to confirm same person
- Discharge summary says "Type 2 Diabetes" but ICD code on physician order says "E11.9" → validates E11.9 IS Type 2 Diabetes → consistent
- Medication list shows Warfarin but diagnosis doesn't include any condition requiring anticoagulation → flag mismatch for clinical review

Layer 6: Completeness Check & Gap Identification

Check extracted data against requirements checklist:

- Patient demographics complete?
- Physician orders present and signed?
- Face-to-face encounter documentation present?
- Insurance information complete?
- Diagnosis and ICD codes present?
- Homebound status documented?
- Medication list present?

Each missing item becomes a specific follow-up task for the Voice Agent.

Layer 7: Confidence Scoring & Routing

Every extracted field gets a confidence score based on extraction method, validation results, and cross-document confirmation:

- **HIGH confidence** — extracted by rules + passed validation + confirmed across documents → auto-populate into intake record
- **MEDIUM confidence** — extracted by LLM vision + passed validation but only appeared on one document → auto-populate but flag for review
- **LOW confidence** — failed validation + correction uncertain → don't populate, add to gap list for Voice Agent to verify on call

---

Features

Feature 1: 24/7 Inbound Call Handling

The agency's Twilio phone number is always answered by the AI agent — nights, weekends, holidays. No more voicemails, no more missed referrals. The agent detects the caller type (provider vs family vs patient) and adapts the conversation flow accordingly.

Provider call handling:

- Structured clinical intake conversation
- Real-time eligibility checking during the call
- Immediate accept/decline/need-more-info response
- Specific ask for missing documents
- SMS/email confirmation sent to the provider after the call

Family call handling:

- Compassionate, guided conversation in plain language
- Captures what the family knows (patient name, condition, what happened, insurance if available)
- Explains the process and what to expect
- Sets realistic expectations
- Schedules coordinator follow-up
- Never promises coverage or availability

Patient self-referral handling:

- Patient-appropriate pace and language
- Handles confused or elderly callers
- Explains that a physician order is needed for skilled care
- Offers to help coordinate with their doctor
- Captures contact information for follow-up

Feature 2: Intelligent Fax Processing

Fax referral packets are automatically processed through the 7-layer extraction pipeline. The system reads the document, extracts structured data, validates it, catches errors, corrects what it can, and flags what it can't. Within minutes of a fax arriving, the system knows: who is this patient, what do they need, can we serve them, and what's missing.

Feature 3: Real-Time Eligibility Engine

Every referral — whether from a call or a fax — is checked against the agency's actual capabilities in real-time:

- **Service area check** — do we serve this zip code?
- **Insurance check** — do we accept this payer and plan?

- **Clinical matching** — does the patient's diagnosis map to a service type we offer, and does it require certifications we have?
- **Caregiver availability** — do we have a caregiver with the right certifications, in the right area, available within the required timeframe?
- **Coverage rules** — does this insurance require prior authorization? What documentation is needed?

The result is one of three outcomes:

- **ACCEPT** — we can take this patient, here's the matched caregiver, here's what documentation we still need
- **DECLINE** — we can't serve this patient (wrong area, wrong insurance, no caregiver available), respond fast so the referral source can place the patient elsewhere
- **NEEDS_MORE_INFO** — we might be able to accept but need specific missing information before deciding

Feature 4: Automated Outbound Follow-up

When the document pipeline or eligibility engine identifies gaps, the Intake Agent automatically triggers outbound actions:

Outbound voice call to referring provider:

- "We received the referral for Mrs. Johnson. We can accept but we're missing the face-to-face encounter documentation. Can you send that to our fax?"
- If no answer, leave a structured voicemail and send SMS follow-up

Outbound voice call to patient/family:

- "Hi Mrs. Johnson, I'm calling from ABC Home Health. Your doctor has referred you for home nursing visits. I'd like to confirm a few details and get your first visit scheduled."
- Confirm address, schedule preference, emergency contact

SMS/email confirmations:

- Confirmation to the referring provider with referral status
- Document request links for missing paperwork
- Appointment confirmation to patient/family

Retry logic:

- If call goes to voicemail, schedule retry in 2 hours
- If SMS gets no response, follow up next morning
- Escalate to human coordinator after 3 failed contact attempts

Feature 5: Caller Personalization

The system recognizes returning callers and personalizes the experience:

- A discharge planner who calls regularly gets a streamlined experience — "Hi Sarah, calling from Mount Sinai? Go ahead with the referral details."
- The system remembers referral source history — how many referrals they've sent, acceptance rate, preferred communication method
- Previous interactions inform the conversation — if a family member called yesterday about their mom, and the coordinator is following up today, the agent has the full context

Feature 6: Guardrails & Compliance

Clinical guardrails:

- Never gives medical advice
- Never confirms a caregiver assignment before eligibility is verified
- Flags clinically suspicious data (medication dosages outside normal ranges, conflicting diagnoses)
- Enforces HIPAA-appropriate conversation boundaries

Operational guardrails:

- Never promises admission before the Eligibility Agent confirms
- Always informs callers that a human coordinator will review the decision
- Escalates to a human when confidence is low or the situation is complex
- Maintains audit trail of every decision and the data that informed it

Data guardrails:

- Confidence scoring ensures low-confidence data is never auto-populated
- Cross-document validation catches inconsistencies before they enter the system
- All extracted data is traceable back to the source document and page

Feature 7: Intake Dashboard

A real-time dashboard showing:

- All active referrals and their pipeline status (new → processing → eligible → accepted/declined)
- Extracted data per referral with confidence scores (green/yellow/red)
- Gap list — what's missing and what actions have been taken to fill each gap
- Call transcripts for every voice interaction
- Caregiver match results showing which caregivers were considered and why each was selected or ruled out
- Referral source analytics — which hospitals send the most referrals, acceptance rates, average response times
- Time-to-decision metrics — how fast are we responding to each referral

---

Data Model

Patient Data

- Name, date of birth, gender
- Address (street, city, state, zip code)
- Phone number
- Emergency contact (name, phone, relationship)
- Primary language spoken

Clinical Data

- Primary diagnosis with ICD-10 code
- Secondary diagnoses with ICD-10 codes
- Surgical procedures performed
- Medications (name, dosage, frequency)
- Allergies
- Functional limitations (mobility, self-care ability)
- Homebound status (yes/no, reason — required for Medicare)
- Hospital admission date
- Hospital discharge date

Physician Data

- Ordering physician name
- Physician NPI number
- Physician phone and fax
- Physician specialty
- Face-to-face encounter date
- Signed physician orders (care type ordered, visit frequency, duration)

Insurance Data

- Payer name (Medicare, Medicaid, Humana, Aetna, UnitedHealthcare, etc.)
- Plan type (Part A, Part B, Medicare Advantage, PPO, HMO)
- Member ID
- Group number
- Policy effective date
- Prior authorization required (yes/no)
- Authorization number (if already obtained)

Referral Source Data

- Referring facility name (hospital, SNF, physician office)
- Referring facility type
- Discharge planner or social worker name
- Their phone, fax, email

- EHR system they use
- Referral date and time

Care Request Data

- Service types needed (skilled nursing, PT, OT, speech therapy, HHA)
- Visit frequency requested (3x/week, daily, etc.)
- Duration of care (30 days, 60 days, ongoing)
- Urgency level (routine, urgent, start of care within 24 hours)
- Special instructions from physician

Agency Configuration Data

- Service areas (list of zip codes served)
- Insurance contracts (which payers and plans accepted)
- Service types offered
- Operating hours
- Agency fax number, phone number

Caregiver Roster Data

- Name
- Caregiver type (RN, LPN, CNA, PT, OT, Speech Therapist, HHA)
- Certifications held (wound care, IV therapy, pediatric, cardiac, orthopedic, diabetes education, etc.)
- Certification expiry dates
- Service zip codes (where willing to work)
- Availability schedule (days, hours)
- Languages spoken
- Current patient load
- Max patient capacity
- Status (active, on leave, suspended)

Knowledge & Reference Data

- ICD-10 code table (code, description, hierarchy) — ~70,000 codes, subset of top 30 home health diagnoses for hackathon
- SNOMED CT to ICD-10 mappings — from National Library of Medicine
- Diagnosis → service type mappings (hip replacement → skilled nursing + PT)
- Service type → certification requirement mappings (skilled nursing → RN or LPN)
- Payer coverage rules (Medicare Part A → covers skilled nursing, requires homebound status, requires F2F encounter, no prior auth)
- RxNorm medication reference (drug names, standard dosage ranges, interactions)
- NPI registry (for physician validation — free CMS API)

Pipeline & Operational Data

- Intake record status (new, processing, pending_documents, eligible, accepted, declined)
- Extracted fields per document with confidence scores (high/medium/low)
- Gap list (what's missing, what actions taken to fill, current status)
- Call transcripts (full text from Twilio Conversational Intelligence)
- Follow-up actions (SMS sent, call attempted, voicemail left, document received, callback scheduled)
- Timestamps for every event (referral received, extraction complete, eligibility checked, first call made, patient admitted)

Database Architecture

Why 4 databases?

Each data type has different access patterns. Using one database for everything would mean compromising on performance for at least one critical path.

PostgreSQL — Operational Data

All structured, relational, transactional data:

- Intake records with status tracking
- Extracted field data with confidence scores (stored as JSONB)
- Caregiver roster
- Service area configuration
- Insurance contracts
- Referral source directory
- Follow-up event logs
- Call transcripts
- Audit trails
- Reference/lookup tables (ICD-10 codes, zip codes, medication ranges)

Why PostgreSQL: ACID transactions (critical for intake status updates — can't have two agents accepting the same patient), complex multi-constraint queries for caregiver matching (cert + zip + availability + load), JSONB for flexible document extraction output.

Neo4j — Knowledge Graph

Relationship-rich data that requires traversal:

13 Node Types:

- Patient
- Diagnosis (ICD-10)
- Payer
- Insurance Plan

- Coverage Rule
- Service Type (skilled nursing, PT, OT, speech, HHA)
- Certification Type
- Caregiver
- Service Area (zip codes)
- Physician
- Referral Source (hospital, SNF, physician office)
- Medication
- Document (extracted pages from referral packet)

14 Relationship Types:

- Patient → HAS_DIAGNOSIS → Diagnosis
- Patient → HAS_INSURANCE → Insurance Plan
- Insurance Plan → UNDER_PAYER → Payer
- Insurance Plan → COVERS → Service Type (with conditions like visit limits, homebound requirement)
- Insurance Plan → REQUIRES_AUTH → Service Type (with rules)
- Diagnosis → REQUIRES → Service Type
- Service Type → NEEDS_CERTIFICATION → Certification Type
- Caregiver → HAS_CERTIFICATION → Certification Type (with expiry date)
- Caregiver → SERVES_AREA → Service Area
- Physician → ORDERED_CARE → Patient
- Referral Source → SENT_REFERRAL → Patient
- Medication → PRESCRIBED_FOR → Diagnosis
- Medication → CONTRAINDICATED_WITH → Medication
- Document → MENTIONS → Patient / Physician / Diagnosis

Why Neo4j: The eligibility check is a path traversal — "does a valid path exist from this patient's diagnosis through required service types through required certifications to an available caregiver in their area with valid insurance coverage?" That's 6 hops. In SQL that's a monster query with 5-6 joins. In Cypher it's a clean traversal. The diagnosis → certification → caregiver matching, insurance hierarchy (payer → plan → state → service → rule), and ICD-10 code hierarchy are all naturally graph-shaped.

Data sources for the graph:

- ICD-10 hierarchy — publicly available from CMS, loaded as nodes with parent-child relationships
- SNOMED CT to ICD-10 mappings — from National Library of Medicine crosswalk
- RxNorm drug data — from NLM, for medication validation and interaction checking
- NPI registry — free CMS API for physician validation
- CMS Medicare home health rules — well-defined enough to manually encode as graph relationships
- Insurance coverage rules — simulated but realistic for hackathon (4-5 payers, 2-3 plans each)
- Diagnosis → service → certification mappings — hand-built for top 30 home health diagnoses

Redis — Pipeline State & Caching

Ephemeral, high-frequency read/write data:

- Document pipeline state (which layer is each page on, what's been extracted so far, validation status)
- Real-time call state (what data has been collected during an active call, what questions remain)
- Eligibility check result caching (same zip + insurance combo checked 5 minutes ago, reuse result)
- Follow-up retry scheduling (call back this number in 2 hours)
- Rate limiting for Twilio calls

Why Redis: Pipeline state changes rapidly as a document moves through 7 layers. PostgreSQL can handle this but Redis is faster for this read-heavy, write-heavy, short-lived pattern. Also natural for caching and scheduling.

pgvector (PostgreSQL extension) — Fuzzy Matching

- Insurance plan name matching (fax says "Humana Gold" but database has "Humana Gold Plus HMO")
- Medication name variation matching (brand names vs generics, abbreviations)
- Physician name matching across documents
- Future: RAG over payer policy documents for edge-case insurance questions

Why pgvector over a separate vector DB: It's a PostgreSQL extension, not a separate database. Same connection, same transactions. For the hackathon scope, PostgreSQL's built-in pg_trgm (trigram similarity) may be sufficient — pgvector is a nice-to-have.

Tech Stack

- **Backend:** FastAPI (Python)
- **Agent Orchestration:** LangGraph
- **LLM:** Gemini Flash (entity extraction, agentic review, conversation reasoning)
- **Telephony:** Twilio ConversationRelay (voice), Twilio Programmable SMS
- **Speech:** Twilio-integrated STT/TTS (or Deepgram/ElevenLabs via ConversationRelay)
- **OCR/Document Parsing:** Docling (text-layer PDFs) + Gemini Flash vision (image PDFs)
- **Databases:** PostgreSQL, Neo4j, Redis, pgvector
- **Frontend Dashboard:** React (simple status dashboard for demo)
- **Infrastructure:** Docker Compose (PostgreSQL, Neo4j, Redis, FastAPI, React dev server)

Intake Workflow — End to End

Flow 1: Discharge Planner Calls the Agency

Phone rings

→ Twilio routes to WebSocket server

→ Voice Agent picks up in PROVIDER MODE

→ "Thank you for calling ABC Home Health, how can I help you?"

→ Planner: "I have a discharge referral — 72-year-old female, hip replacement..."

→ Voice Agent extracts data in real-time (name, DOB, diagnosis, insurance, zip)

→ Passes to Intake Agent orchestrator

→ Orchestrator sends to Eligibility Agent

→ Eligibility Agent traverses Neo4j + PostgreSQL

→ Service area? YES

→ Insurance accepted? YES (Medicare Part A)

→ Diagnosis → service mapping? Hip replacement → skilled nursing + PT

→ Certification requirements? RN with ortho cert + licensed PT

→ Available caregivers? 3 RNs, 2 PTs in that zip

→ Coverage rules? No prior auth, needs F2F doc + homebound cert

→ Returns: ACCEPT, missing F2F encounter note

→ Orchestrator tells Voice Agent what to say

→ Voice Agent: "We can accept this referral, we have availability within 48 hours.

We'll need the face-to-face encounter documentation — can you send that?"

→ Call ends

→ Orchestrator triggers Follow-up Agent

→ SMS confirmation to planner with referral ID

→ Intake record created in PostgreSQL with status PENDING_DOCUMENTS

→ Gap: F2F encounter note — follow up in 4 hours if not received

Flow 2: Fax Referral Arrives

Fax arrives as PDF

→ Document Pipeline Layer 1: preprocesses (deskew, denoise)

→ Layer 2: classifies each page (discharge summary, physician order, insurance card, etc.)

→ Layer 3: routes each page to appropriate extraction path (rules or vision LLM)

→ Layer 4: extracts entities into raw JSON

→ Layer 5: agentic review loop

→ Validation Agent checks all fields

→ Correction Agent fixes what it can with confidence scores

→ Cross-Reference Agent checks consistency across pages

→ Layer 6: completeness check — identifies gaps

→ Layer 7: confidence scoring — routes fields to auto-populate / flag / gap list

→ Structured data passes to Intake Agent orchestrator

→ Orchestrator sends to Eligibility Agent

→ Eligibility check runs (same as Flow 1)

→ Result: ACCEPT but missing F2F note + low-confidence insurance member ID

→ Orchestrator triggers actions:

→ Voice Agent makes OUTBOUND CALL to referring provider

→ "We received the referral for Mrs. Johnson. We can accept.

Could you verify the insurance member ID? We have H120456789.

Also we need the face-to-face encounter note."

→ Voice Agent makes OUTBOUND CALL to patient/family

→ "Hi, I'm calling from ABC Home Health. Your doctor has referred you

for home nursing visits. I'd like to confirm your address and

schedule your first visit."

→ SMS/email sent with document upload link

→ Intake record tracks all gaps and follow-up status

Flow 3: Family Member Calls

Phone rings at midnight

→ Twilio routes to WebSocket server

→ Voice Agent picks up in FAMILY MODE

→ "Thank you for calling ABC Home Health. I'm here to help. What's going on?"

→ Daughter: "My mom just had a stroke, she's in the hospital, they said she'll need home care when she comes home, I don't know where to start"

→ Voice Agent uses compassionate tone, simple language

→ Asks gentle questions: mom's name, which hospital, approximate zip code, what kind of help she thinks she'll need, does she have insurance info handy

→ Captures what's available (may be incomplete — that's okay)

→ Passes to Intake Agent orchestrator

→ Orchestrator does a PRELIMINARY eligibility check

→ Can we serve that zip code? YES

→ Do we generally accept that insurance type? YES

→ Can we handle stroke recovery care? YES

→ Orchestrator tells Voice Agent what to say

→ Voice Agent: "Based on what you've told me, we should be able to help. A care coordinator will call you tomorrow morning to discuss the details. Could you have your mom's insurance card and her doctor's contact information ready? Is there a best time to reach you?"

→ Call ends

→ Orchestrator creates intake record with status NEW

→ Flags for human coordinator follow-up by 9 AM

→ SMS to daughter: "Thank you for calling ABC Home Health. A coordinator will contact you tomorrow morning. If you have questions before then, call us anytime."

Hackathon Build Plan

Pre-work (Before Hackathon Day)

All data prep and architecture decisions — zero application code:

- PROJECT.md (this document)
- .cursorrules with stack decisions, conventions, file structure
- PHASE_N_SPEC.md for each phase
- PostgreSQL migration SQL ready
- Neo4j Cypher seed scripts ready (ICD-10 subset, diagnosis-certification mappings, insurance rules)
- Sample referral PDFs (3-4 with varying completeness and quality)
- Caregiver roster data as JSON (20-30 caregivers)
- Insurance rules data as JSON (4-5 payers, 2-3 plans each)
- Twilio account created, phone number provisioned
- Docker Compose file ready

Phase 1: Foundation (All 4 people, Hour 1)

- Person 1: FastAPI skeleton with health checks, WebSocket endpoint for Twilio
- Person 2: Document pipeline scaffolding (file upload endpoint, processing queue)
- Person 3: PostgreSQL schema migration + Neo4j seed data loading
- Person 4: Twilio account configuration, ConversationRelay setup, Redis setup

Phase 2: Knowledge & Eligibility Engine (Person 3, Hours 2-3)

- Eligibility Agent implementation
- Neo4j traversal queries (diagnosis → service → certification → caregiver matching)
- PostgreSQL queries (service area, insurance contract, caregiver availability)
- Accept / decline / needs-more-info decision logic
- API endpoint: POST /eligibility-check → returns decision with reasons

Phase 3: Document Pipeline (Person 2, Hours 2-3)

- PDF ingestion and preprocessing

- Page classification (LLM-based)
- Entity extraction (rules for clean PDFs, Gemini vision for messy ones)
- Validation Agent (format checks, code lookups, range checks)
- Correction Agent (re-examine source, reason about errors, confidence scoring)
- Cross-Reference Agent (consistency checks across pages)
- Completeness check and gap identification
- API endpoint: POST /process-document → returns structured JSON with gaps

Phase 4: Voice Agent (Person 1, Hours 2-3)

- Twilio ConversationRelay WebSocket handler
- Provider mode conversation flow
- Family mode conversation flow
- Outbound mode conversation flow
- Real-time data extraction from caller speech
- Integration with Eligibility Agent for mid-call checks
- Call transcript capture

Phase 5: Orchestrator (Person 4, Hours 2-3)

- LangGraph state machine defining the full intake workflow
- State: referral received → document processing → eligibility check → decision → follow-up
- Routing logic: document pipeline output triggers eligibility, eligibility triggers voice actions
- Follow-up Agent: SMS via Twilio, email, retry scheduling
- Merge logic: combine data from document pipeline + voice conversations into unified intake record

Phase 6: Dashboard & Demo (All 4 people, Hours 4-5)

- Person 3: React dashboard — intake pipeline view, extracted data with confidence scores, gap tracker
- Person 4: Wire all components together, end-to-end integration testing
- Person 1: Live call testing, edge case handling, demo script prep
- Person 2: Connect document pipeline to orchestrator, test with sample PDFs
- All: Final 30 minutes — rehearse demo, prepare scripted scenarios, test edge cases

Demo Script

1. **Live inbound call from discharge planner** — judge calls the Twilio number, gives referral details, gets a real-time answer on the spot
2. **Fax processing** — upload a sample referral PDF, show it processing through 7 layers on the dashboard, watch confidence scores populate, see gaps identified
3. **Outbound follow-up** — system automatically calls a number to collect missing documentation
4. **Family call** — judge calls as a worried family member, gets a compassionate intake experience

5. **Dashboard** — show the full pipeline: all referrals, their status, extracted data, confidence scores, gap lists, call transcripts, caregiver matches

---

Judging Criteria Alignment

Reliability — The agentic validation-correction loop ensures data accuracy. Confidence scoring prevents bad data from entering the system. Retry logic handles failed calls and missed follow-ups.

Guardrails — Voice Agent never gives medical advice, never confirms admission without eligibility verification, flags clinically suspicious data, escalates to humans when uncertain, maintains HIPAA-appropriate conversation boundaries.

Security — All patient data stored in encrypted databases. Call transcripts handled per HIPAA requirements. No PHI exposed in logs or dashboard without authentication. Twilio provides HIPAA-eligible telephony infrastructure.

Scalability — Modular agent architecture means each component scales independently. Document pipeline can process multiple faxes in parallel. Voice Agent can handle concurrent calls via Twilio's infrastructure. Redis caching prevents redundant eligibility checks.

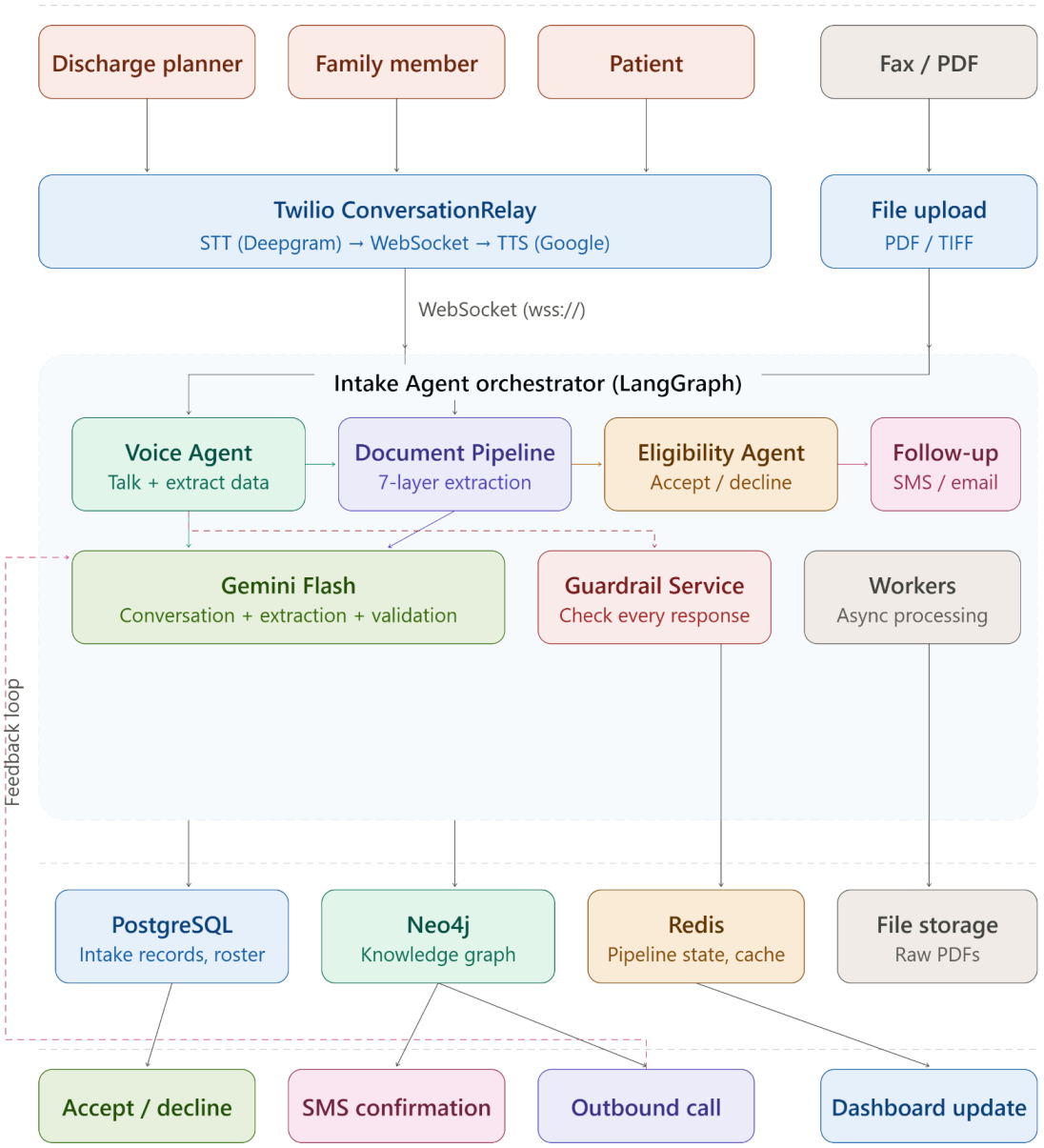
Domain Knowledge — Real ICD-10 codes, real SNOMED CT mappings, real NPI validation, real Medicare coverage rules. Not an LLM guessing — a knowledge graph grounded in actual clinical and insurance data.

Caller Personalization — System recognizes caller type and adapts conversation flow. Remembers referral source history. Maintains context across follow-up interactions.

Why This Matters to Arya Health

This is literally their next product. Arya has 5 agents managing the caregiver lifecycle (supply side) but nothing handling the patient intake lifecycle (demand side). The Intake Agent was announced as their top priority during their Series A in late 2025 but has not shipped publicly. Building a working prototype that demonstrates the full workflow — inbound calls, fax processing, real-time eligibility checking, outbound follow-up — shows the Arya engineering team that we understand their domain, their architecture, and their product roadmap.

IntakeAI — full system architecture



l = voice / Neo4j | Purple = docs | Amber = eligibility / Redis | Pink = follow-up | Green = Gemini / output | Red = guardrails

PHASE 2: Guardrails & Conversation Boundaries

Objective

Define and implement every safety rule the system enforces — what the Voice Agent is allowed to say, what it must never say, when to escalate to a human, how confident it needs to be before acting, and how to resolve conflicting data from multiple sources. These rules are enforced at two levels: prompt-level (soft, LLM follows instructions) and code-level (hard, blocked before reaching the caller).

This phase produces no API endpoints, no database queries, no UI. It produces the rules, the service that enforces them, the prompt templates that embed them, and the tests that prove they work. Everything built in Phases 3-5 depends on these rules being defined first.

Why This Phase Exists Separately

The hackathon judges explicitly evaluate "reliability, guardrails, security, and scalability" as prerequisites, not extras. If the Voice Agent says "you're admitted" before the Eligibility Agent confirms, or gives medical advice to a worried daughter, or reads back a full insurance member ID on a phone call — that's a demo-ending failure. Defining the rules before writing the application code means every engineer knows the boundaries from the start.

Owner

No single owner — this phase is pre-work that feeds into every other phase. The guardrail service is used by Engineer 1 (Voice Agent), the prompts are used by Engineers 1 and 2, the confidence thresholds are used by Engineers 2 and 3, and the data merge rules are used by Engineer 4. Ideally done as pre-work or very early in Hour 1.

Core Principle: Rules as Config, Not Code

All guardrail rules — blocked patterns, safe replacements, confidence thresholds, escalation triggers, data merge priorities — live in a JSON configuration file under `data/`. The GuardrailService reads this file at startup. To add a new blocked pattern during the hackathon, edit the JSON, restart the server. No code changes needed.

Similarly, all system prompts live as plain text files in `backend/prompts/`. The Voice Agent loads the right prompt file based on the conversation mode. To adjust the agent's tone or add a guardrail instruction, edit the text file. No code changes needed.

Deliverable 1: Guardrail Configuration File

File: `data/guardrail_rules.json`

This JSON file is the single source of truth for all enforceable rules. It has four sections:

Section 1: Blocked Patterns

An array of objects, each defining a regex pattern that must never appear in outgoing Voice Agent messages, along with the reason it's blo

cked and a safe replacement phrase.

Patterns to include:

Full SSN exposure — any pattern that looks like a Social Security Number being read aloud (three digits, two digits, four digits with the SSN/social security keywords nearby). Reason: HIPAA violation. Replacement: the agent should never be reading SSNs at all, so the replacement is a redirect like "I'm not able to share that information over the phone."

Medical advice language — patterns where the agent tells the caller what medication to take, what treatment to pursue, or what they "should" do medically. Phrases like "you should take," "you must take," "you need to take" followed by a medication or treatment. Reason: the agent is an intake coordinator, not a clinician. Replacement: "I'd recommend discussing that with your doctor" or "Our clinical team can address that once care begins."

Premature admission confirmation — phrases like "you are admitted," "you are accepted," "you are approved" stated as fact before the Eligibility Agent has confirmed. Reason: legal liability — if the agent says "you're accepted" and then the eligibility check fails, the agency has a problem. Replacement: "Based on our initial review, we should be able to help" or "Our coordinator will confirm the details."

Guarantee language — the word "guarantee" or phrases promising specific outcomes. Reason: no guarantee can be made about healthcare outcomes or service delivery. Replacement: remove the guarantee and state what the agency aims to do instead.

Specific caregiver promises — "your nurse will be [name]" or "your caregiver is [name]." Reason: the agent doesn't assign caregivers — the Staffing Agent does, and assignments can change. Replacement: "We'll match you with a qualified nurse in your area."

Specific date/time promises — "your first visit is on [date]" or "your appointment is [time]." Reason: the agent doesn't schedule visits — scheduling happens after admission. Replacement: "We aim to start care within 48 hours of completing the referral."

Full insurance ID readback — reading back a complete insurance member ID on the phone. Reason: sensitive data that could be overheard. Safe behavior: confirm only the last 4 digits. Replacement: "I have a member ID ending in [last 4]."

Diagnosis disclosure to non-providers — if the agent is in family mode and tries to state specific clinical details from a fax that the family hasn't mentioned. Reason: the family member may not have full clinical context, and the agent shouldn't be the one delivering clinical information. Replacement: "Your mom's doctor has provided us with the clinical details. Our care coordinator can discuss the care plan with you."

Each pattern object should have: `name` (human-readable identifier), `pattern` (the regex), `reason` (why it's blocked), `replacement` (the safe alternative text), and `severity` (critical vs warning — critical blocks the message entirely, warning logs but allows with modification).

Section 2: Confidence Thresholds

An object mapping threshold names to numeric values. These thresholds control how the system behaves at different confidence levels:

eligibility_confirm_threshold (default: 0.7) — the minimum confidence in the eligibility result before the Voice Agent can say "we can accept." Below this threshold, the agent says "I'll need to have our coordinator verify this and get back to you."

field_auto_populate_threshold (default: 0.5) — the minimum confidence for an extracted field to be auto-populated into the intake record. Below this, the field goes on the gap list and the Voice Agent asks the caller to confirm.

field_reject_threshold (default: 0.3) — below this confidence, the extracted value is discarded entirely. Not populated, not flagged — thrown away. It's too unreliable to even show.

correction_accept_threshold (default: 0.8) — when the Correction Agent fixes an OCR error, the correction is auto-accepted only above this confidence. Between 0.5 and this threshold, the correction is applied but flagged for review. Below 0.5, the correction is retried with enriched context.

caller_identification_threshold (default: 0.8) — how confident Gemini needs to be about whether the caller is a provider, family member, or patient before committing to a conversation mode. Below this, ask a clarifying question rather than guessing.

All thresholds are configurable. During the hackathon, if the system is being too conservative or too aggressive, adjust the number in the JSON and restart.

Section 3: Escalation Triggers

An array of trigger definitions. Each trigger describes a condition that, when detected, causes the Voice Agent to offer a human coordinator or end the automated session.

Triggers to define:

human_request — caller explicitly asks to speak to a person, a human, a coordinator, a manager, or says "I want to talk to someone real." Action: acknowledge the request, say a

coordinator will be in touch, end the ConversationRelay session gracefully, create a high-priority follow-up task.

caller_distress — caller is crying, shouting, expressing extreme frustration, or using distress language. Detection: Gemini identifies emotional distress in the transcript. Action: acknowledge the emotion ("I understand this is a stressful time"), offer a human coordinator, but don't force the escalation — the caller might prefer to continue.

repeated_misunderstanding — the agent has failed to understand the caller 3 or more times in a row (asking them to repeat, getting unclear responses, caller saying "no that's not what I said"). Action: apologize for the difficulty, offer to connect with a coordinator who can help directly.

clinical_question — caller asks a question that requires clinical judgment: "Will my mom be able to walk again?" "Is this medication safe?" "What's the prognosis?" Action: redirect to clinical staff. "That's a great question for our clinical team. I'll make sure your care coordinator addresses that."

max_call_duration — call has exceeded 15 minutes. Action: begin wrapping up. "I want to make sure I'm respecting your time. I have all the key details. Let me summarize what we've covered and our coordinator will follow up on the remaining items."

off_topic_persistent — caller keeps steering the conversation to unrelated topics after gentle redirects. Action: after 2-3 redirects, offer to schedule a separate call or connect with a coordinator.

data_conflict_unresolvable — during a call, the caller provides information that directly contradicts what's in the intake record from a fax, and the agent can't determine which is correct. Action: flag both values, note the conflict, schedule a human review.

Each trigger object should have: `name`, `description` (what it detects), `detection_method` (keyword-based, LLM-inferred, count-based, or time-based), `action` (what the agent does), and `priority` (how urgently the follow-up should be handled).

Section 4: Data Merge Priority Rules

An array of rules that define which data source wins when the same field has different values from a fax extraction and a voice conversation (or two different calls).

Rules to define:

contact_info_caller_wins — for fields: patient phone number, patient address, emergency contact name and phone, patient language preference. The caller (voice conversation) wins over fax data. Reason: the caller has current information; the fax might contain outdated hospital records.

clinical_data_fax_wins — for fields: ICD-10 codes, physician orders, medication list (names and dosages), diagnosis details, procedures performed. The fax (clinical document) wins over caller-provided data. Reason: clinical documents are authoritative; a family

member saying "she had a hip operation" is less precise than the discharge summary saying "Z96.641 — right artificial hip joint."

insurance_flag_conflict — for fields: payer name, plan type, member ID, group number. If the fax and caller provide different insurance information, do NOT auto-resolve. Flag the conflict, keep both values, add to gap list for human review. Reason: wrong insurance information means denied claims. The risk of picking the wrong value is worse than the delay of human verification.

patient_name_keep_complete — if the fax says "Johnson, Maria" and the caller says "Maria L. Johnson-Smith," keep the more complete version ("Maria L. Johnson-Smith") and flag for review. Do not silently pick one.

physician_data_fax_wins — for fields: physician name, NPI, phone, fax, orders. The fax is the authoritative source for physician information. The caller might not know their physician's NPI or the exact order details.

default_keep_both — for any field not covered by a specific rule, keep both values, flag the conflict, and add to gap list for human resolution. Never silently discard data.

Each merge rule object should have: `name`, `fields` (array of field names it applies to), `winner` (caller, fax, or flag), `reason`, and `action` (what to do: overwrite, keep_both, flag_for_review).

Git commit: `phase-2: guardrail rules configuration file`

Deliverable 2: System Prompts

Directory: `backend/prompts/`

Seven plain text files, each containing a complete system prompt for a specific Gemini usage context. These are loaded at runtime by the Voice Agent and pipeline agents. No prompt content is hardcoded in Python files — the Python code reads from these text files.

Voice Conversation Prompts

`provider_inbound.txt` — System prompt for when a healthcare provider (discharge planner, physician, SNF coordinator) calls.

This prompt should establish:

- The agent's identity: "You are an intake coordinator at ABC Home Health Agency."
- The agent's goal: collect all necessary referral information to make an admit/decline decision.
- The conversation style: professional, clinical, efficient. Use medical terminology naturally. Don't over-explain things a healthcare provider already knows.

- The data collection checklist: patient name, date of birth, diagnosis/procedure, insurance (payer AND plan type), zip code, physician name and NPI, care types needed, visit frequency, discharge date, urgency level.
- The JSON response format: every response must be a JSON object with "response" (what to say), "extracted" (structured data pulled from this turn), "needs_clarification" (fields to ask about next), and "ready_for_eligibility" (boolean — true when diagnosis + insurance + zip are all collected).
- What NOT to do: never give medical advice, never confirm admission before receiving an eligibility result from the system, never guarantee a specific caregiver or start date, never read back full insurance member IDs.
- How to handle eligibility results: when a system message arrives with eligibility results, communicate them naturally — "We can accept this referral" or "Unfortunately we don't serve that area" — without mentioning "the system" or "the database."
- How to handle missing information: if the caller hasn't provided a required field, ask for it directly. Providers expect structured questions. "What's the patient's date of birth?" not "Could you maybe tell me when they were born?"

`family_inbound.txt` — System prompt for when a family member calls.

Different tone entirely:

- The agent's identity: same agency, but the personality is warm, patient, reassuring.
- The conversation style: compassionate, uses simple language, avoids medical jargon. Explain things the way you would to someone who has never dealt with home health care before. Speak slowly and clearly.
- The data collection approach: gentle, not interrogative. "Can you tell me a little about what happened?" not "What is the primary diagnosis?" Collect what the family knows — they won't have ICD codes or NPI numbers. That's okay.
- What the family typically knows: patient's name, what happened (surgery, stroke, fall), which hospital, approximate location, insurance company (maybe). What they don't know: ICD codes, physician NPI, specific care orders, homebound status.
- Setting expectations: "Based on what you've told me, we should be able to help. A care coordinator will call you tomorrow morning to discuss the details. Could you have your mom's insurance card and her doctor's contact information ready?"
- What NOT to do: never give medical advice, never explain the patient's condition, never promise coverage, never promise a specific start date, never say "you're accepted" — only that "we should be able to help" pending coordinator review.
- Emotional handling: if the caller is upset, anxious, or crying, acknowledge it. "I understand this is a stressful time. We're here to help." Don't rush them. Don't pivot immediately to data collection — let them talk for a moment, then gently guide the conversation.

`outbound_followup.txt` — System prompt for outbound calls the agent makes.

This is the most structured prompt because the agent has a specific mission:

- The agent is calling a known person (referring provider, patient, or family member) about a known referral.
- The prompt should include placeholders for: the referral ID context, who we're calling (name, role), what we need from them (specific missing documents, data to verify, schedule to confirm), and what we already know (so the agent doesn't re-ask questions already answered).
- The conversation style: brief, respectful of the recipient's time, clear about what's needed. "I'm calling from ABC Home Health regarding the referral for Mrs. Johnson. We just need the face-to-face encounter documentation to complete the admission. Could you send that to our fax?"
- Voicemail handling: if the agent detects an answering machine, leave a concise message with the agency name, what's needed, and the callback number. No patient clinical details in the voicemail (HIPAA).
- What NOT to do: same clinical and admission guardrails as the other modes, plus don't disclose patient information to someone who hasn't been verified as the right contact.

Document Pipeline Prompts

`extraction.txt` — System prompt for Gemini vision when extracting data from a document page.

This prompt tells Gemini what to look for on a medical referral document page:

- Extract all identifiable fields into a structured JSON format
- The expected field categories: patient demographics, clinical information, physician details, insurance information, medication list, care orders
- How to handle handwriting: do your best, mark uncertain characters with a confidence flag
- How to handle partial information: extract what's visible, leave missing fields as null
- The output JSON schema the extraction should conform to (field names must match what the rest of the system expects — same field names as the intake_record JSONB columns)

`validation.txt` — System prompt for the Validation Agent.

Tells Gemini to check extracted fields against reference data:

- Validate ICD-10 codes against the provided reference list
- Validate NPI numbers using the Luhn algorithm check digit
- Validate medication dosages against known clinical ranges provided as reference
- Validate zip codes against a US format
- Validate date logic (discharge after admission, DOB not in future, reasonable age range)
- For each field, output: valid (boolean), error description if invalid, severity (critical/warning)

`correction.txt` — System prompt for the Correction Agent.

Tells Gemini to fix extraction errors:

- Given an extracted value that failed validation, the original source text/image context, and reference data showing valid values, determine the most likely correct value
- Assign a confidence score to the correction (0.0 to 1.0)
- Explain the reasoning (e.g., "the character 'l' in M17.1l is likely a '1' based on the font and surrounding digit pattern, correcting to M17.11 which is a valid ICD-10 code")
- If you can't determine the correct value with reasonable confidence, say so — don't guess

`cross_reference.txt` — System prompt for the Cross-Reference Agent.

Tells Gemini to check consistency across multiple pages of the same referral packet:

- Compare patient name across all pages — are variations of the same person or different people?
- Compare diagnosis/ICD codes across discharge summary and physician orders — do they agree?
- Compare insurance information across pages — same member ID, same plan?
- Compare medication lists if they appear on multiple pages — any contradictions?
- For each inconsistency found, output: the field, the conflicting values with page numbers, which value is more likely correct and why, and a recommended resolution

Git commit: `phase-2: system prompts (provider mode, family mode, outbound mode, pipeline agents)`

Deliverable 3: Guardrail Service

File: `backend/services/guardrail_service.py`

A service class that loads the guardrail rules JSON at initialization and exposes methods used by other parts of the system.

What the service does:

On initialization: reads `data/guardrail_rules.json`, compiles all regex patterns from the blocked_patterns section into compiled regex objects (compile once, use many times), and stores the thresholds, escalation triggers, and merge rules in memory.

Method: check_outgoing_message(message_text, conversation_mode)

This is the core method. Called on EVERY message the Voice Agent is about to send to Twilio for TTS, before it's sent.

Takes the raw message text and the current conversation mode (provider, family, outbound). Runs every compiled blocked pattern against the text. Returns one of:

- PASS — no patterns matched, message is safe to send
- BLOCKED — one or more patterns matched. Returns the list of violations (pattern name, reason, matched text) and the suggested replacement. The Voice Agent does NOT send this message. Instead, it feeds the violation back to Gemini with a guardrail correction message (Loop 1 from .cursorrules) asking Gemini to rephrase.

For family mode specifically, apply additional checks: scan for medical terminology that shouldn't be used with non-clinical callers. If Gemini uses terms like "ICD-10 code," "NPI number," "homebound certification," "F2F encounter" in a family-mode conversation, flag it — the family doesn't know what those mean.

Method: check_confidence(field_name, confidence_score)

Takes a field name and its confidence score. Returns one of:

- AUTO_POPULATE — confidence is above the auto-populate threshold
- FLAG_FOR_REVIEW — confidence is between the reject and auto-populate thresholds
- REJECT — confidence is below the reject threshold
- CONFIRM_WITH_CALLER — confidence is below auto-populate but above reject, and the field is critical (insurance ID, diagnosis, patient name) — ask the caller to confirm rather than silently accepting

Method: check_eligibility_confidence(eligibility_result)

Takes the eligibility result from the Eligibility Agent. Evaluates whether the result is confident enough for the Voice Agent to state "we can accept." Factors to consider:

- Did the eligibility check find an exact insurance plan match or a fuzzy match?
- Are all required caregiver certifications actively valid (not expiring within 7 days)?
- Is there at least one caregiver available or is the match based on a caregiver who's almost at capacity?

Returns: CONFIRM (tell the caller we can accept), HEDGE (tell the caller we should be able to help but need to verify), or DEFER (tell the caller a coordinator will review and call back).

Method: check_escalation(conversation_state)

Takes the current conversation state (turn count, caller sentiment indicators, repeated misunderstanding count, call duration, topic keywords). Evaluates each escalation trigger from the config. Returns either NO_ESCALATION or the specific trigger that fired with its prescribed action.

Method: resolve_merge_conflict(field_name, fax_value, caller_value)

Takes a field name and two conflicting values from different sources. Looks up the applicable merge rule from the config. Returns: the winning value, the losing value, and the action (overwrite, keep_both, or flag_for_review). If no specific rule covers this field, applies the default_keep_both rule.

What the service does NOT do:

- It does NOT call Gemini. It's pure rule evaluation — regex matching, threshold comparison, config lookup.
- It does NOT modify messages. It reports violations. The caller (Voice Agent) decides what to do with the report.
- It does NOT access any database. It reads only from the JSON config file loaded at startup.

`ponytail:` one class, no inheritance, no strategy pattern, no plugin architecture for guardrail types. It's a config reader + regex matcher + threshold comparator. Ceiling: adding a new guardrail type (e.g., ML-based toxicity detection) would require code changes, not just config. Upgrade path: add a `custom_checks` plugin mechanism if the project goes beyond hackathon.

Git commit: `phase-2: guardrail service with blocked patterns`

Deliverable 4: Confidence Threshold Configuration

This is already part of the guardrail_rules.json (Section 2), but it needs to be wired into the rest of the system. The confidence thresholds affect three areas:

Document Pipeline (Phase 4): After the agentic review loop produces confidence scores per extracted field, the pipeline calls `guardrail_service.check_confidence()` to decide whether to auto-populate, flag, or reject each field. The pipeline doesn't hardcode threshold values — it asks the guardrail service.

Eligibility Agent (Phase 3): After the eligibility check, before the result is communicated to the Voice Agent, the Eligibility Agent calls `guardrail_service.check_eligibility_confidence()` to decide whether to say "accept," "we should be able to help," or "a coordinator will review."

Voice Agent (Phase 5): During mid-call extraction, the accumulated data fields each have implicit confidence (data from a provider is higher confidence than data from a family member who's unsure). The Voice Agent uses thresholds to decide whether to confirm data with the caller or accept it as stated.

The key design point: thresholds are defined in ONE place (the JSON config), read by ONE service (GuardrailService), and called by multiple consumers. No consumer hardcodes a threshold number.

Git commit: `phase-2: confidence threshold configuration`

Deliverable 5: Escalation Trigger Definitions

Also part of guardrail_rules.json (Section 3), but the detection logic needs to be specified clearly for the Voice Agent to implement.

For each trigger, the detection method is:

human_request: keyword detection in the caller's transcribed speech. Look for phrases like "speak to a person," "talk to someone," "human," "real person," "your manager," "supervisor," "coordinator." This is a regex/keyword check, not an LLM inference.

caller_distress: LLM-inferred. The Gemini system prompt includes an instruction: "If the caller appears distressed, upset, crying, or angry, include 'caller_distress: true' in your JSON response." The WebSocket handler checks for this flag in Gemini's output each turn.

repeated_misunderstanding: count-based. The WebSocket handler tracks how many consecutive turns resulted in the agent asking the caller to repeat or clarify. A counter incremented when Gemini's response contains clarification requests ("Could you repeat that?" "I didn't catch that" "Could you spell that for me?"). Reset to zero when the agent successfully extracts new data. Triggers at 3 consecutive misunderstandings.

clinical_question: LLM-inferred. The Gemini system prompt includes: "If the caller asks a clinical or medical question that requires professional judgment, include 'clinical_question: true' in your JSON response." The Voice Agent handles this by redirecting.

max_call_duration: time-based. The WebSocket handler tracks call start time. At 12 minutes, inject a system message to Gemini: "This call has been going for 12 minutes. Begin wrapping up in the next 2-3 turns." At 15 minutes, the agent proactively summarizes and ends.

off_topic_persistent: count-based. Track how many times Gemini's response includes a redirect ("Let me bring us back to your referral" or similar). At 3 redirects, escalate.

data_conflict_unresolvable: event-based. When the Voice Agent detects that the caller is providing information that contradicts existing data in the intake record AND the merge rule says "flag_for_review," this trigger fires.

Git commit: `phase-2: escalation trigger definitions`

Deliverable 6: Data Merge Rules Implementation

Also part of guardrail_rules.json (Section 4), but the merge logic in the Orchestrator (Phase 5) needs to know how to use these rules.

The Orchestrator calls `guardrail_service.resolve_merge_conflict()` whenever it's merging data from a new source into an existing intake record. This happens in two scenarios:

Scenario A: Fax processed, then provider calls about the same patient. The fax extraction already populated the intake record. Now the provider is on the phone providing additional or different data. For each field the caller provides, the Orchestrator checks: does this field already have a value from the fax? If yes, call resolve_merge_conflict. If the caller wins (e.g., updated phone number), overwrite. If the fax wins (e.g., ICD code), keep the fax value but log that the caller said something different. If it's insurance data, flag for human review.

Scenario B: Family calls first, fax arrives later. The family call captured partial data (name, general condition, approximate zip, insurance company name). Then the fax arrives with detailed clinical data. For each field the fax extracts, the Orchestrator checks: does this field already have a value from the call? If yes, apply the merge rules. Clinical fields from the fax overwrite the family's less precise descriptions. Contact info from the family call is kept over the fax.

Scenario C: Multiple calls about the same patient. A family member calls at night, then a discharge planner calls the next morning about the same patient. The Orchestrator needs to detect that these are about the same patient (name + DOB matching, or referral source matching) and merge the data. The provider call has higher authority for clinical data, the family call has higher authority for current contact information and preferences.

The merge process always produces an audit trail: for every field, store which source provided the value, when it was provided, whether it was overwritten and by what. This audit trail lives in the intake record's JSONB fields as metadata alongside the values.

Git commit: `phase-2: data merge rules implementation`

Deliverable 7: Guardrail Unit Tests

File: `backend/tests/test_guardrails.py`

Tests that prove the guardrail service works correctly. These tests load the same guardrail_rules.json and verify behavior.

Test cases to cover:

Blocked pattern tests:

- A message containing "you are accepted" should return BLOCKED
- A message containing "you are admitted to our program" should return BLOCKED
- A message saying "Based on our review, we should be able to help" should return PASS

- A message containing "I guarantee" should return BLOCKED
- A message containing "your nurse will be Maria" should return BLOCKED
- A message saying "we'll match you with a qualified nurse" should return PASS
- A message containing "you should take Lisinopril" should return BLOCKED
- A message saying "your doctor can discuss medication options" should return PASS
- A message containing a fake SSN pattern (123-45-6789) should return BLOCKED
- A message confirming "member ID ending in 6789" should return PASS (partial ID is safe)

Confidence threshold tests:

- Field with confidence 0.9 → AUTO_POPULATE
- Field with confidence 0.6 → FLAG_FOR_REVIEW
- Field with confidence 0.2 → REJECT
- Critical field (insurance_member_id) with confidence 0.6 → CONFIRM_WITH_CALLER (not just flag)

Merge rule tests:

- Patient phone from caller vs fax → caller wins
- ICD code from caller vs fax → fax wins
- Insurance member ID conflict → flag for review, keep both
- Patient name "Maria Johnson" (fax) vs "Maria L. Johnson-Smith" (caller) → keep the more complete one, flag
- Unknown field with conflict → keep both (default rule)

Escalation trigger tests:

- Conversation state with 3 consecutive misunderstandings → triggers repeated_misunderstanding
- Call duration at 16 minutes → triggers max_call_duration
- Call duration at 10 minutes → no trigger

These tests verify the guardrail service in isolation — no database, no Gemini, no Twilio. Pure unit tests against the config.

Git commit: `phase-2: guardrail unit tests passing`

How Phase 2 Connects to Other Phases

Phase 3 (API): The EligibilityService calls `guardrail_service.check_eligibility_confidence()` before returning results.

Phase 4 (Document Pipeline): The confidence scoring layer (Layer 7) calls `guardrail_service.check_confidence()` to route each field to auto-populate / flag / reject.

Phase 5 (Voice Agent): The WebSocket handler calls `guardrail_service.check_outgoing_message()` on every response before sending to Twilio. The Orchestrator calls `guardrail_service.resolve_merge_conflict()` when merging data. The conversation handler checks `guardrail_service.check_escalation()` after every turn.

Phase 6 (Dashboard): The dashboard reads confidence scores and gap flags that were set using the guardrail thresholds. The color coding (green/yellow/red) on the dashboard directly maps to the threshold ranges defined in this phase.

What NOT to Build in Phase 2

- No API endpoints — the guardrail service is called internally by other services, not exposed via HTTP
- No database access — the service reads from a JSON file only
- No Gemini calls — guardrails are deterministic rules, not LLM inference (except escalation triggers that are detected by Gemini, but that detection happens in the Voice Agent, not in the guardrail service)
- No Twilio integration — the service checks messages, it doesn't send them
- No complex NLP for pattern detection — regex is sufficient for the blocked patterns. The LLM handles the nuanced detection (distress, clinical questions) in its own response, and the guardrail service just reads those flags.

PHASE 3: API & Backend Core Services

Objective

Build the FastAPI application with all HTTP endpoints, WebSocket stubs, SQLAlchemy models, Pydantic schemas, and the core services (Eligibility, Caregiver Matching, Intake lifecycle, Document management, Follow-up). After this phase, every service can be called via HTTP, the Eligibility Agent can traverse Neo4j and PostgreSQL to make admit/decline decisions, and the Voice Agent (Phase 5) and Document Pipeline (Phase 4) have a backend to talk to.

Owner

Engineer 3 (primary for Eligibility Engine, API endpoints, caregiver matching) during Hours 2-3. **Engineer 1** scaffolds the FastAPI skeleton and WebSocket endpoint during Hour 1.

Core Principle: Services Contain Logic, Routes Are Thin

Every API route file in `backend/api/` is a thin layer — it validates the request (via Pydantic), calls a service, and returns the response. No business logic in routes. All business logic lives in `backend/services/`. This way, the Voice Agent (Phase 5) and Background Workers (Phase 4) can call the same services directly without going through HTTP.

Phase 1 Alignment Note: SQLAlchemy 2.0

Phase 1 spec described `database.py` as exporting raw asyncpg/neo4j/redis clients. We've since decided on SQLAlchemy 2.0. This phase updates `database.py` to include a SQLAlchemy async engine and session factory alongside the Neo4j driver and Redis client. The raw SQL schema from Phase 1 (`postgres_init.sql`) still runs on container start — the SQLAlchemy models in `tables.py` mirror that same schema, giving engineers autocomplete, type safety, and a living schema reference.

Deliverable 1: SQLAlchemy Models

File: `backend/models/tables.py`

SQLAlchemy 2.0 declarative models that mirror every table defined in the Phase 1 PostgreSQL schema. These models are the Python representation of the database schema — every engineer reads this file to understand what columns exist, what types they are, and what relationships connect them.

What each model needs:

Every model should:

- Use SQLAlchemy 2.0 `DeclarativeBase` with `MappedAsDataclass` or `Mapped[]` type annotations — this gives type hints that Cursor's autocomplete understands
- Map to the exact table name from postgres_init.sql (snake_case)
- Include all columns with their types matching the SQL schema
- Use `mapped_column()` with appropriate SQLAlchemy types: UUID for ids, String for text, Enum for enum types, JSON for JSONB columns, DateTime for timestamps, Boolean for flags, Integer for numbers, ARRAY for PostgreSQL arrays
- Define foreign key relationships where they exist

Models to create, matching Phase 1's schema exactly:

IntakeRecord — maps to `intake_records`. This is the most important model. All JSONB columns (patient_data, clinical_data, physician_data, insurance_data, care_request, referral_source, extraction_confidence, gaps, eligibility_reasons, matched_caregivers) should be typed as `JSON` in SQLAlchemy, which maps to PostgreSQL JSONB. The status and source fields use SQLAlchemy Enum types matching the Phase 1 enums.

Caregiver — maps to `caregivers`. The `languages` column is a PostgreSQL ARRAY of strings — use SQLAlchemy's `ARRAY(String)` type. The `type` and `status` fields use enums.

CaregiverCertification — maps to `caregiver_certifications`. Include the foreign key relationship to Caregiver. Note: the `is_active` column is a PostgreSQL GENERATED column — in SQLAlchemy, map it as a regular Boolean column with `server_default` or mark it as computed. The database manages it; the ORM just reads it.

CaregiverServiceArea — maps to `caregiver_service_areas`. Foreign key to Caregiver.

CaregiverAvailability — maps to `caregiver_availability`. Foreign key to Caregiver. The `start_time` and `end_time` columns use SQLAlchemy's `Time` type.

ServiceArea — maps to `service_areas`. Primary key is `zip_code` (string), not a UUID.

InsuranceContract — maps to `insurance_contracts`.

ReferralSource — maps to `referral_sources`. The `acceptance_rate` is a GENERATED column — same treatment as CaregiverCertification's `is_active`.

Document — maps to `documents`. Foreign key to IntakeRecord. The `processing_status` field uses an enum.

DocumentPage — maps to `document_pages`. Foreign key to Document.

CallRecord — maps to `call_records`. Foreign key to IntakeRecord. The `twilio_call_sid` has a unique constraint.

FollowUpAction — maps to `follow_up_actions`. Foreign key to IntakeRecord.

Relationships to define:

- IntakeRecord has many Documents (one-to-many)
- IntakeRecord has many CallRecords (one-to-many)
- IntakeRecord has many FollowUpActions (one-to-many)
- Document has many DocumentPages (one-to-many)
- Caregiver has many CaregiverCertifications (one-to-many)
- Caregiver has many CaregiverServiceAreas (one-to-many)
- Caregiver has many CaregiverAvailabilities (one-to-many)

These relationships let SQLAlchemy eagerly or lazily load related objects — e.g., loading an IntakeRecord can include its documents and call records in one query.

`ponytail:` no abstract base model class, no mixin classes for timestamps, no custom column type wrappers. Just flat model classes. The `updated_at` trigger is handled by the database (Phase 1's SQL trigger), not the ORM. Ceiling: no automatic `updated_at` in Python-only tests. Upgrade: add an `onupdate` parameter to the column if needed.

Git commit: `phase-3: sqlalchemy models for all tables`

Deliverable 2: Updated Database Connections

File: `backend/models/database.py`

Update the Phase 1 database.py to include SQLAlchemy alongside Neo4j and Redis.

The file should export:

SQLAlchemy async engine and session:

- Create an async engine using `create_async_engine` from sqlalchemy.ext.asyncio, with the asyncpg driver (the connection string becomes `postgresql+asyncpg://...`)
- Create an async session factory using `async_sessionmaker`
- Export a dependency function that yields an async session (for use in FastAPI dependency injection)

Neo4j async driver:

- Same as Phase 1 — AsyncGraphDatabase.driver initialized from settings
- Export `get_neo4j()` returning the driver

Redis async client:

- Same as Phase 1 — redis.asyncio.from_url initialized from settings
- Export `get_redis()` returning the client

Lifecycle functions:

- `init_all_dbs()` — creates the SQLAlchemy engine, initializes Neo4j driver, connects Redis. Called on FastAPI startup.
- `close_all_dbs()` — disposes engine, closes driver and client. Called on FastAPI shutdown.

`ponytail:` three clients in one file, no separate module per database, no connection manager class. Ceiling: all three clients share one file. Upgrade: split into `postgres.py`, `neo4j_client.py`, `redis_client.py` if the file gets unwieldy.

Git commit: `phase-3: database connections updated for sqlalchemy`

Deliverable 3: Pydantic Schemas

File: `backend/models/schemas.py`

Pydantic models for all API request and response bodies. Every endpoint receives and returns Pydantic models, never raw dicts. This gives automatic validation, serialization, and OpenAPI documentation.

Schemas to define:

Intake schemas:

- `IntakeRecordCreate` — request body for creating a new intake record. Required: source (enum). Optional: urgency, patient_data, clinical_data, physician_data, insurance_data, care_request, referral_source (all as dict/Any since they're JSONB).
- `IntakeRecordUpdate` — request body for updating. All fields optional. Includes status changes, data updates, eligibility results.
- `IntakeRecordResponse` — response body. All fields from the database model. Includes id, status, source, all JSONB data fields, extraction_confidence, gaps, eligibility_decision, matched_caregivers, timestamps. Used for both single record and list responses.
- `IntakeRecordList` — response body for listing. Array of IntakeRecordResponse plus a count.

Eligibility schemas:

- `EligibilityCheckRequest` — what the Eligibility Agent needs: diagnosis (icd_code or text description), insurance_payer, insurance_plan (optional — might not be known yet), zip_code, service_types_needed (optional — can be inferred from diagnosis).
- `EligibilityCheckResponse` — the full result: decision (accept/decline/needs_more_info), reasons (array of reason objects), matched_caregivers (array of caregiver match objects with scores and match reasons), missing_documents (array of required document types not yet received), coverage_details (prior auth required, visit limits, episode duration), confidence_score.

Caregiver schemas:

- `CaregiverMatchRequest` — requirements for matching: certification_types needed, zip_code, day_of_week and time (optional), language (optional).
- `CaregiverMatchResponse` — array of matching caregivers, each with: id, name, type, certifications (filtered to relevant ones), distance/area match, availability match, current load vs capacity, overall match score, reasons for ranking.

Document schemas:

- `DocumentUploadResponse` — after upload: document id, file_name, page_count, processing_status.
- `DocumentStatusResponse` — current processing state: id, status, current_layer (1-7), extraction_result (if complete), confidence_scores, gaps.
- `DocumentPageResponse` — per-page detail: page_number, classification, extraction_path, extraction data, confidence scores, validation errors.

Call schemas:

- `CallRecordResponse` — call details: id, twilio_call_sid, direction, mode, caller_number, status, transcript, extracted_data, duration, timestamps.

Follow-up schemas:

- `FollowUpActionCreate` — request to schedule a follow-up: type, target_phone or target_email, message, scheduled_at.
- `FollowUpActionResponse` — action details: id, type, status, target, message, scheduled_at, executed_at, result, attempt_number.

Shared schemas:

- `StatusUpdate` — simple request body for status changes: new_status, reason (optional).
- `ErrorResponse` — standard error format: detail (string), error_code (optional).

`ponytail:` all schemas in one file. No separate file per domain. No schema inheritance hierarchy. Ceiling: one large file. Upgrade: split into `intake_schemas.py`, `eligibility_schemas.py`, etc. if the file exceeds 300 lines.

Git commit: `phase-3: pydantic schemas for all request/response models`

Deliverable 4: FastAPI Application Skeleton

File: `backend/main.py`

The FastAPI application entry point. This file does:

- Create the FastAPI app instance with title "IntakeAI", description, and version
- Register the startup event to call `init_all_dbs()`
- Register the shutdown event to call `close_all_dbs()`
- Include all API routers from `backend/api/` with appropriate prefixes and tags
- Configure CORS middleware (allow all origins for hackathon — the frontend runs on a different port)
- Add a root health check endpoint that returns the service status and database connectivity

Router registration with prefixes:

- `/api/intake` — intake record CRUD
- `/api/documents` — document upload and status
- `/api/eligibility` — eligibility checks
- `/api/caregivers` — caregiver matching
- `/api/followup` — follow-up actions
- `/voice` — Twilio webhooks and WebSocket (no /api prefix — Twilio hits these directly)

`ponytail:` no middleware for request logging, no middleware for request timing, no global exception handlers beyond FastAPI's defaults. Add them only if a specific debugging need arises during the hackathon. Ceiling: no structured request logging. Upgrade: add a simple middleware if debugging becomes painful.

Git commit: `phase-3: fastapi skeleton with health check endpoint`

Deliverable 5: Intake Endpoints

File: `backend/api/intake.py`

CRUD endpoints for intake records. These are thin — they validate input, call IntakeService, and return the response.

POST `/api/intake` — Create a new intake record. Receives IntakeRecordCreate. The IntakeService creates the row in PostgreSQL with status "new" and returns the record with its generated UUID. This is called when: a new fax arrives, a new inbound call starts, or a manual referral is entered.

GET `/api/intake/{id}` — Get a single intake record by ID. Returns the full IntakeRecordResponse with all JSONB data, confidence scores, gaps, eligibility result, matched caregivers, and related data. The IntakeService loads the record from PostgreSQL.

PUT `/api/intake/{id}/status` — Update the status of an intake record. Receives StatusUpdate (new status + optional reason). The IntakeService validates the state transition

(e.g., can't go from "new" directly to "accepted" — must pass through "eligible" first), updates the record, and returns the updated record. This is called by the Orchestrator as the referral moves through the pipeline.

PUT `/api/intake/{id}` — Update data fields on an intake record. Receives IntakeRecordUpdate with any combination of JSONB data fields. The IntakeService merges the new data into existing JSONB (not replaces — merges). This is called after each voice conversation turn extracts new data, or after the document pipeline extracts fields.

GET `/api/intake` — List all intake records, optionally filtered by status. Returns IntakeRecordList. Supports query parameters: `status` (filter by status), `since` (only records created after this timestamp), `limit` and `offset` for pagination. The dashboard polls this endpoint every 3 seconds.

File: `backend/services/intake_service.py`

The business logic for intake record lifecycle:

- Creating records with proper defaults
- Validating state transitions (define which status transitions are allowed)
- Merging JSONB data (when new data arrives from a call or document, merge it into existing fields without overwriting the entire JSONB column — use PostgreSQL's JSONB concatenation operator or Python-level merge)
- Updating timestamps and tracking which source provided which data

The merge logic should use the data merge rules from the GuardrailService (Phase 2) when the new data might conflict with existing data.

Git commit: `phase-3: intake endpoints (create, get, update status, list active)`

Deliverable 6: Eligibility Service

File: `backend/services/eligibility_service.py`

This is the most critical service in the system. It takes patient data and determines: can we serve this patient?

What the service does, step by step:

Step 1: Service area check (PostgreSQL)

Query the `service_areas` table: does a row exist for the patient's zip code with active=true? This is a simple primary key lookup. If no → DECLINE, reason: "We do not serve zip code {zip}." Stop here.

Step 2: Insurance check (PostgreSQL + Neo4j)

First, query `insurance_contracts` in PostgreSQL: do we have an accepted contract for this payer and plan? If the caller provided a specific plan (e.g., "Medicare Part A"), check for an exact match. If they only provided the payer name (e.g., "Medicare"), check if we accept ANY plan under that payer and note which ones.

If no matching contract → DECLINE, reason: "We do not accept {payer} {plan}." Stop here.

If we have a match, query Neo4j for the coverage rules: traverse InsurancePlan → COVERS → ServiceType to get the coverage details (prior auth required? what documents needed? visit limits?). Store this for the response.

Step 3: Clinical matching (Neo4j)

This is the graph traversal that justifies Neo4j.

Given the patient's diagnosis (ICD-10 code or text description), traverse the graph:

Diagnosis → REQUIRES → ServiceType: what service types does this diagnosis need? This might return multiple services (e.g., hip replacement requires skilled_nursing + physical_therapy).

For each required ServiceType → NEEDS_CERTIFICATION → CertificationType: what certifications does a caregiver need to provide this service? Note the "either" flag — skilled nursing needs RN OR LPN (either qualifies), while physical therapy specifically needs a PT.

Also check: does the REQUIRES relationship have a `specialization` property? If the diagnosis specifically needs wound care, the caregiver needs the wound_care certification IN ADDITION to the base RN/LPN certification.

If the diagnosis isn't in Neo4j (unknown ICD code or text description that doesn't match) → NEEDS_MORE_INFO, reason: "Unable to determine service requirements for this diagnosis. Coordinator review needed." Don't decline — just flag.

Step 4: Caregiver availability (PostgreSQL)

Now query PostgreSQL to find caregivers who match ALL requirements:

- Have an active certification matching the needed CertificationType (join caregiver_certifications where is_active=true)
- Cover the patient's zip code (join caregiver_service_areas)
- Status is "active" (not on_leave or suspended)
- Current patient load is below max capacity
- Optionally: available on the likely first visit day (join caregiver_availability)
- Optionally: speak the patient's language if specified

This is a multi-join query with several WHERE filters. Order results by: match quality (exact specialty cert > base cert only), proximity (same zip vs adjacent zip), current load (less loaded caregiver preferred).

If zero caregivers match → NEEDS_MORE_INFO, reason: "No available caregivers with required certifications in that area. Coordinator will check for alternatives." Don't decline — the agency might have a caregiver who can adjust their schedule.

If caregivers match → ACCEPT (pending document requirements).

Step 5: Assemble the result

Combine all findings into the EligibilityCheckResponse:

- decision: ACCEPT, DECLINE, or NEEDS_MORE_INFO
- reasons: array explaining each factor that contributed to the decision
- matched_caregivers: ranked list of qualifying caregivers with match scores
- coverage_details: from Neo4j — prior auth required, required documents, visit limits
- missing_documents: compare what the coverage rules require vs what the intake record already has
- confidence_score: how confident the system is in this result

Before returning, call `guardrail_service.check_eligibility_confidence()` to determine if the Voice Agent should state this confidently or hedge.

Performance requirement:

This entire sequence (Steps 1-5) must complete in under 3 seconds. The caller is on the phone. Steps 1 and 2 (PostgreSQL lookups) are sub-millisecond. Step 3 (Neo4j traversal) should be under 100ms for a graph with ~250 nodes. Step 4 (caregiver matching) depends on how many caregivers exist — with 25 sample caregivers, it's sub-millisecond. The bottleneck will be the sequential nature, not any single query.

`ponytail:` run steps sequentially, not in parallel. The total is under 500ms for hackathon data volumes. Parallel execution adds complexity for negligible time savings. Ceiling: at production scale (1000+ caregivers, 500+ diagnoses), parallel queries matter. Upgrade: use asyncio.gather() for steps 2+3 and step 4.

Git commit: `phase-3: eligibility service (neo4j traversal + postgres queries)`

Deliverable 7: Eligibility Endpoint

File: `backend/api/eligibility.py`

POST `/api/eligibility/check` — Receives EligibilityCheckRequest, calls EligibilityService, returns EligibilityCheckResponse. This endpoint is called by the Voice Agent mid-call (via the Orchestrator) and by the Document Pipeline after extraction completes.

GET `/api/eligibility/service-areas` — Returns the list of served zip codes. Used by the dashboard and potentially by the Voice Agent to quickly confirm "yes we serve Brooklyn" without a full eligibility check.

GET `/api/eligibility/insurance` — Returns the list of accepted insurance contracts. Same purpose.

Git commit: `phase-3: eligibility endpoint wired to service`

Deliverable 8: Caregiver Match Service and Endpoint

File: `backend/services/caregiver_match_service.py`

Separated from the Eligibility Service because caregiver matching can also be called independently — e.g., the dashboard wants to show which caregivers would match a referral without re-running the full eligibility check.

The service takes requirements (certification types, zip code, optional day/time, optional language) and queries PostgreSQL to find matching caregivers. Same multi-join query as Step 4 of the Eligibility Service, but exposed as a standalone service.

Each matched caregiver gets a match score calculated from:

- Certification match: exact specialty match (e.g., wound_care cert for a wound care patient) scores higher than base certification match (e.g., just RN)
- Area match: same zip code scores higher than adjacent zip
- Load: caregiver at 2/8 capacity scores higher than one at 7/8
- Availability: available on the likely first visit day scores higher than "probably available"
- Language: matching patient's preferred language is a bonus

Return caregivers ranked by score with the scoring breakdown visible (so the dashboard can show "why this caregiver was recommended").

File: `backend/api/caregivers.py`

POST `/api/caregivers/match` — Receives CaregiverMatchRequest, calls CaregiverMatchService, returns ranked list of matching caregivers.

GET `/api/caregivers/{id}` — Get a single caregiver's full details including certifications, service areas, and availability. Used by the dashboard detail view.

GET `/api/caregivers` — List all caregivers with basic info. Used by the dashboard.

Git commit: `phase-3: caregiver match service and endpoint`

Deliverable 9: Document Upload Endpoint

File: `backend/api/documents.py`

POST `/api/documents/upload` — Receives a file upload (PDF or TIFF). The endpoint saves the file to disk (in a local uploads directory), creates a Document row in PostgreSQL with status "uploaded," and returns the DocumentUploadResponse with the document ID. It does NOT start processing — that's the background worker's job (Phase 4). It does create a background task (FastAPI BackgroundTasks) to notify the document processor that a new file is ready.

Optionally accepts an `intake_record_id` query parameter to link the document to an existing intake record. If not provided, the document is unlinked until the Orchestrator associates it.

GET `/api/documents/{id}/status` — Returns the current processing state of a document. The Dashboard polls this to show pipeline progress.

GET `/api/documents/{id}/extraction` — Returns the full extraction result with per-page details, confidence scores, and validation errors. Only available when processing_status is "complete."

File: `backend/services/document_service.py`

Handles document lifecycle:

- Saving uploaded files to disk
- Creating and updating Document rows in PostgreSQL
- Linking documents to intake records
- Querying document status and extraction results

The actual extraction logic (7-layer pipeline) lives in Phase 4. This service just manages the database records and file storage.

Git commit: `phase-3: document upload endpoint`

Deliverable 10: Follow-up Action Endpoints

File: `backend/api/followup.py`

POST `/api/followup` — Schedule a new follow-up action. Receives FollowUpActionCreate. The FollowUpService creates the row in PostgreSQL and, if the

action has a scheduled_at time, adds it to the Redis sorted set for the scheduler worker to pick up.

GET `/api/followup/{intake_id}` — List all follow-up actions for a given intake record. Returns chronological event log. The dashboard uses this to show the follow-up timeline for a referral.

PUT `/api/followup/{id}/status` — Update a follow-up action's status (e.g., mark as completed, failed, or cancelled).

File: `backend/services/followup_service.py`

Handles follow-up lifecycle:

- Creating follow-up action records
- Adding scheduled actions to Redis sorted set (score = scheduled_at timestamp, value = action ID)
- Sending SMS via Twilio Programmable SMS API
- Updating action status after execution
- Implementing retry logic: if an action fails, create a new action with attempt_number incremented and scheduled_at pushed back by the retry interval

The actual Twilio SMS sending logic lives here. The service needs the Twilio client (initialized from settings, using the twilio Python SDK). For the hackathon, email sending can be stubbed — just log "would send email to X" and mark as completed. SMS is the priority.

Git commit: `phase-3: follow-up action endpoints`

Deliverable 11: Voice Webhook and WebSocket Stubs

File: `backend/api/voice.py`

These are stubs in Phase 3 — the actual Voice Agent logic is Phase 5. But the endpoints need to exist so Twilio has something to hit.

POST `/voice/inbound` — Twilio webhook for incoming calls. Returns TwiML XML that points to ConversationRelay with the WebSocket URL (ngrok URL + /voice/stream), the welcome greeting, and TTS/STT provider configuration. This endpoint is called by Twilio when someone dials the agency number.

WebSocket `/voice/stream` — The ConversationRelay WebSocket endpoint. In Phase 3, this is a stub that: accepts the connection, receives the setup event, logs it, and sends a simple text response for any prompt event ("Thank you for calling. Our system is currently being set up. Please call back shortly."). Phase 5 replaces this stub with the full conversation handler.

POST `/voice/test` — Text-based testing endpoint (bypasses Twilio). Accepts JSON with a message and a session_id. Maintains conversation state in memory per session. Returns the agent's response as JSON. This endpoint is critical for rapid iteration — test the conversation logic without making phone calls.

POST `/voice/outbound` — Trigger an outbound call. Accepts the target phone number and the call mission (what to say, what data to collect). Uses the Twilio REST API to initiate the call with TwiML pointing to ConversationRelay. In Phase 3, this is a stub that creates the Twilio call but the ConversationRelay handler is the same Phase 3 stub.

Git commit: `phase-3: voice webhook and websocket endpoints (stubs)`

Deliverable 12: Integration Test

Run through the full API surface with sample requests to verify everything connects.

Tests to run manually or via a simple test script:

1. **Health check:** GET `/` returns service info and database connectivity status
2. **Create intake record:** POST `/api/intake` with source "inbound_call_provider" → returns a record with UUID and status "new"
3. **Update intake data:** PUT `/api/intake/{id}` with patient_data and clinical_data → returns merged record
4. **Eligibility check — accept scenario:** POST `/api/eligibility/check` with diagnosis Z96.641, insurance Medicare Part A, zip 11201 → returns ACCEPT with matched caregivers
5. **Eligibility check — decline (zip):** POST `/api/eligibility/check` with zip 90210 → returns DECLINE, reason: not in service area
6. **Eligibility check — decline (insurance):** POST `/api/eligibility/check` with Aetna HMO (not accepted per sample data) → returns DECLINE, reason: insurance not accepted
7. **Caregiver match:** POST `/api/caregivers/match` with certification RN + orthopedic, zip 11201 → returns ranked list
8. **Document upload:** POST `/api/documents/upload` with a sample PDF → returns document ID with status "uploaded"
9. **Schedule follow-up:** POST `/api/followup` with type sms_sent → returns action record
10. **Voice webhook:** POST `/voice/inbound` → returns valid TwiML XML

If all 10 pass, Phase 3 is complete.

Git commit: `phase-3: all endpoints tested with sample requests`

How Phase 3 Connects to Other Phases

Phase 1 → Phase 3: Phase 3 reads from the databases Phase 1 set up. The SQLAlchemy models mirror the Phase 1 SQL schema. The Neo4j queries in the Eligibility Service traverse the graph Phase 1 seeded.

Phase 2 → Phase 3: The Eligibility Service calls `guardrail_service.check_eligibility_confidence()` before returning results. The Intake Service calls `guardrail_service.resolve_merge_conflict()` when merging data from multiple sources.

Phase 3 → Phase 4: The Document upload endpoint creates the record; the Phase 4 background worker picks it up and processes it. The Document Service provides the database operations the worker needs.

Phase 3 → Phase 5: The Voice Agent calls the Eligibility endpoint mid-call. The Orchestrator calls Intake endpoints to create/update records, Follow-up endpoints to schedule actions, and Caregiver Match to find available nurses. The voice webhook stubs get replaced with real handlers.

Phase 3 → Phase 6: The dashboard polls the Intake list endpoint every 3 seconds. The detail view calls the Intake get endpoint, Document status endpoint, and Follow-up list endpoint. The live call monitor reads from the Call Record data.

What NOT to Build in Phase 3

- No LangGraph orchestration — that's Phase 5
- No document processing pipeline — that's Phase 4
- No real voice conversation handling — Phase 5 replaces the stubs
- No background workers — Phase 4
- No frontend — Phase 6
- No authentication/authorization — `ponytail:` no auth for hackathon, everything is localhost. Ceiling: anyone on the network can hit the API. Upgrade: add API key middleware or JWT if deploying beyond hackathon.
- No rate limiting — `ponytail:` not needed for a demo with 1-2 concurrent users
- No request logging middleware — add it only if debugging becomes painful
- No pagination optimization (cursor-based) — `ponytail:` limit/offset is fine for 50 records max during demo